Informe DevOps

DOCENTE
Cesar Augusto Palacios Alarcon

INTEGRANTES

Elkin Mauricio Idarraga Valencia

2026

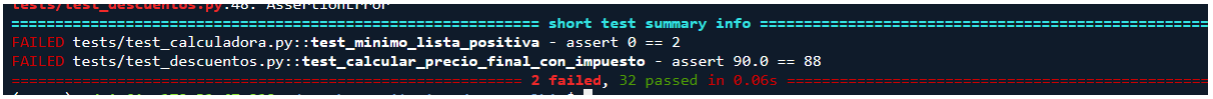
Pruebas unitarias en formato AAA

Módulo: Calculadora

Para el módulo de calculadora se realizaron **19 pruebas unitarias** enfocadas en verificar el funcionamiento de las operaciones matemáticas y el comportamiento de las funciones ante diferentes tipos de entradas. En las operaciones aritméticas se probaron números positivos y negativos, resultados iguales a cero y operaciones con resultados enteros. Para la potencia se evaluaron exponentes positivos y el exponente cero, mientras que en la raíz cuadrada se comprobó el cálculo con números que tienen una raíz exacta.

En las funciones de promedio, máximo y mínimo se utilizaron listas con diferentes valores, incluyendo números negativos y listas vacías cuando correspondía. Estos casos permiten comprobar que los resultados se calculen correctamente y que las funciones respondan de forma adecuada ante entradas límite. También se evaluó el manejo de excepciones en la división por cero y en los cálculos de área y perímetro cuando se proporcionan dimensiones negativas.

La selección de los casos se realizó teniendo en cuenta las operaciones principales de cada función, las condiciones definidas en el código y los posibles escenarios en los que podrían presentarse resultados incorrectos. Las pruebas siguen el patrón Arrange, Act, Assert (AAA), lo que facilita identificar los datos utilizados, la operación ejecutada y el resultado esperado. De esta manera, se busca comprobar el comportamiento funcional del módulo y detectar inconsistencias sin repetir innecesariamente escenarios equivalentes, mas adelante en el informe veremos el porcentaje de cobertura alcanzado con estas pruebas.



Como podemos observar en la imagen anterior, todos los test escritos para el modulo calculadora pasaron menos uno, sin embargo, esto se debe a que la funcion minimo incluye un bug a propósito, lo que hace que la prueba falle, mas adelante nos encargaremos de eso.

Módulo: Descuentos

Para este módulo se realizaron 22 pruebas unitarias, con el objetivo de verificar el funcionamiento de las operaciones de descuento y el manejo de valores inválidos. Los casos se seleccionaron teniendo en cuenta las diferentes funciones, las

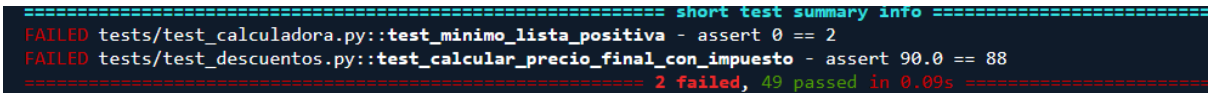
condiciones de validación y los principales escenarios de uso, buscando una cobertura representativa sin repetir pruebas innecesariamente.

Las pruebas se organizaron de la siguiente manera:

- **Aplicación y cálculo de descuentos (6 pruebas):** se verificó el cálculo de un descuento normal, un descuento del 100 %, precios negativos y porcentajes fuera del rango permitido. Se eligieron estos casos para comprobar tanto los resultados esperados como las restricciones de entrada.
- **Cálculo del precio final con impuesto (4 pruebas):** se evaluó el cálculo del precio con descuento e impuesto, además del manejo de precios negativos, impuestos negativos y porcentajes de descuento inválidos. Estos escenarios permiten comprobar las validaciones de la función y su comportamiento ante diferentes entradas.
- **Descuentos por volumen (2 pruebas):** se probaron una cantidad que no genera descuento y otra que activa el descuento del 5 %. Se seleccionaron estos casos para verificar dos condiciones distintas de la escala de descuentos por cantidad.
- **Descuentos acumulativos (5 pruebas):** se comprobó la aplicación de un descuento, la aplicación sucesiva de varios descuentos, una lista vacía y el rechazo de porcentajes negativos o superiores al 100 %. Estos casos permiten evaluar el procesamiento de listas y las validaciones realizadas durante el cálculo.
- **Descuentos apilados (3 pruebas):** se verificó el comportamiento con una lista vacía, con un único descuento y con descuentos cuya suma supera el 100 %. Se eligieron estos escenarios para revisar el cálculo conjunto de porcentajes y los límites esperados de la operación.

En conjunto, las **22 pruebas** cubren escenarios normales, valores límite, entradas inválidas y diferentes condiciones de las funciones del módulo. El criterio principal fue comprobar que los cálculos produjeran los resultados esperados y que las restricciones definidas se aplicaran correctamente. Los resultados de ejecución y el porcentaje de cobertura obtenido lo analizaremos mas adelante

Veamos como nos fue ejecutando las pruebas:



Tal como en el modulo anterior, todas las pruebas del modulo pasaron correctamente, sin embargo una de las relacionadas con el precio final con impuesto no paso de la manera prevista, esto se debe al bug que hay en el codigo del que luego nos encargaremos

Módulo: Conversor

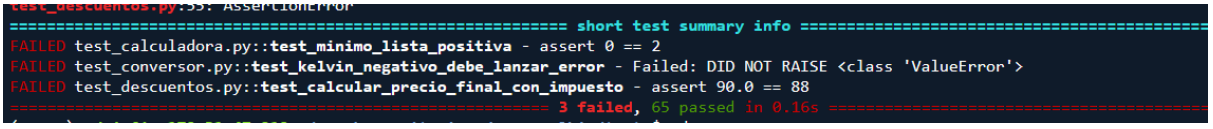
Para este módulo se definieron **27 pruebas unitarias** con el propósito de verificar las conversiones entre diferentes unidades de temperatura, distancia, peso y volumen, así como las validaciones implementadas para rechazar valores negativos. Los casos se seleccionaron considerando el funcionamiento de las distintas funciones, las condiciones de validación y la detección de comportamientos incorrectos.

Las pruebas se organizaron de la siguiente manera:

- **Conversión de temperaturas (5 pruebas):** se evaluaron las conversiones de Celsius a Fahrenheit, Fahrenheit a Celsius, Celsius a Kelvin y Kelvin a Celsius. También se incluyó un caso con una temperatura negativa en Kelvin, con el fin de comprobar si la función rechaza valores inferiores al cero absoluto.
- **Conversión de distancias (10 pruebas):** se verificaron las conversiones entre metros y kilómetros, millas y kilómetros, kilómetros y millas, y pies y metros. Se incluyeron valores válidos y negativos para comprobar tanto los resultados de las operaciones como las restricciones de entrada.
- **Conversión de peso (4 pruebas):** se probaron las conversiones entre libras y kilogramos, así como entre onzas y gramos. Para cada conversión se evaluó un caso válido y uno con un valor negativo, con el objetivo de comprobar la precisión del cálculo y el funcionamiento de las validaciones.
- **Conversión de volumen (4 pruebas):** se evaluaron las conversiones entre galones y litros, y entre litros y galones. Se utilizaron casos válidos y valores negativos para comprobar los resultados y verificar el rechazo de entradas no permitidas.

En conjunto, las **27 pruebas** contemplan escenarios de funcionamiento normal, validaciones de entrada y un caso diseñado específicamente para detectar un defecto en la conversión de Kelvin a Celsius. El criterio de selección fue cubrir las funciones del módulo y sus principales condiciones de ejecución, evitando repetir casos equivalentes con diferentes valores sin una justificación adicional.

A continuación se muestran los resultado de ejecucion de pytest, donde todos los tests del modulo pasan la prueba excepto uno, el cual nos indica que hay un bug en la funcion de pasar de kelvin a celsius, mas adelante analizaremos esto.



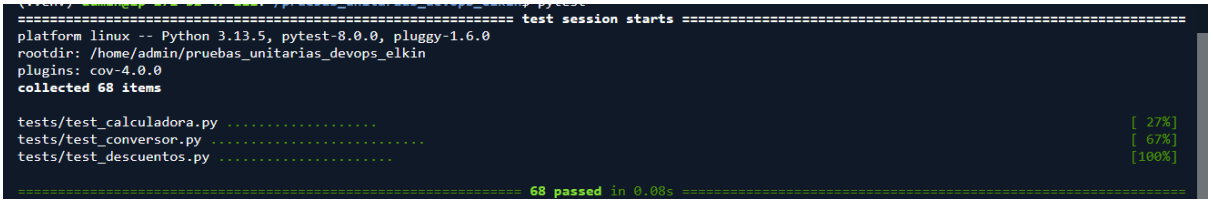
Identificación de Bugs

Como podimos ver anteriormente, en la realización de las pruebas, encontramos que hay 3 que fallan, esta información reveladora nos deja claro que existe algún bug o error en algunas funciones pertenecientes a los módulos que analizamos.

A continuación veremos una tabla resumen del bug encontrado, la entrada que lo causa y la solución propuesta

Modulo	Descripción	Funcion	Entrada	Resultado Erroneo	Resultado Esperado	Correccion aplicada
Calculadora	La función toma como menor valor el 0, lo que hace que al compararlo con otro número positivo este jamas cambie	minimo()	lista = [2,3,4,5] o cualquier lista	minimo = 0	minimo = 2	Se establece que en vez de tomar el valor inicial como 0, se tomara el primer elemento de la lista min_val = lista[0]
Descuentos	La funcion actualmente calcula de manera erronea el impuesto, lo que hace fallar el precio final.	calcular_precio_final()	Precio: 100, impuesto: 10 %, descuento: 20 %	precio = 90	precio = 88	Calcular el impuesto sobre el precio después de aplicar el descuento. impuesto_monto = precio_con_descuento * (impuesto / 100)
Conversor	La funcion no valida que el valor de los grados kelvin sea mayor a cero	kelvin_a_celsius()	-10 kelvins o cualquier valor negativo	-283.15 celsius	ValueError	Validar que la temperatura en Kelvin sea mayor o igual a cero antes de convertir

Luego de solucionar los 3 bugs, volvemos a ejecutar las pruebas mediante pytest y obtenemos lo siguiente:



Cobertura de los módulos seleccionados

Al ejecutar el comando *pytest tests/ -v --cov=src --cov-report=term-missing* obtendremos la cobertura total por modulos que hacen nuestras pruebas, este fue nuestro resultado:

```
---------- coverage: platform linux, python 3.13.5-final-0 -----------
Name                        Stmts   Miss  Cover   Missing
---------------------------------------------------------
src/__init__.py                 0      0   100%
src/autenticacion.py           42     42     0%   6-77
src/calculadora.py             42      2    95%   42, 52
src/conversor.py               58      3    95%   93-95
src/csv_parser.py              76     76     0%   6-127
src/descuentos.py              58      7    88%   51, 54, 60, 62, 64, 75, 92
src/fibonacci.py               42     42     0%   6-73
src/manipulador_texto.py       93     93     0%   6-156
src/procesador_texto.py        49     49     0%   6-84
src/utils.py                   53     53     0%   6-92
src/validador.py               64     64     0%   6-117
---------------------------------------------------------
TOTAL                         577    431    25%
```

Como podemos observar, los tres módulos trabajados superan el 85 % de cobertura: calculadora.py alcanza un 95 %, conversor.py un 95 % y descuentos.py un 88 %. Estos resultados muestran que las pruebas unitarias ejecutan una gran parte del código de los módulos seleccionados, incluyendo operaciones principales, validaciones y diferentes escenarios de entrada.

El módulo de calculadora y el conversor presentan los porcentajes más altos, mientras que descuentos tiene una cobertura ligeramente menor debido a algunas líneas que todavía no se ejecutan durante las pruebas. Estas líneas pendientes pueden revisarse para determinar si corresponden a condiciones adicionales o casos que requieren nuevas pruebas.

Es importante aclarar que el 25 % de cobertura total corresponde al proyecto completo, incluyendo otros módulos que no se trabajaron en esta etapa.. En conclusión, los resultados permiten evidenciar una cobertura adecuada de los módulos evaluados y aportan información para identificar posibles áreas de mejora.

Puntos extra

1. **Prueba para csv parser usando mocking:**

Se diseñó una prueba unitaria para la función `parse_csv_line()`, utilizando una línea CSV que contiene una coma dentro de un campo entre comillas: `"Pérez, Juan",20`. El objetivo es comprobar que la función interprete correctamente los campos y devuelva dos valores: `"Pérez, Juan"` y `"20"`.

Al ejecutar la prueba con la implementación original, esta falla porque la función utiliza `split(',')`, que separa la línea por cada coma sin tener en cuenta las comillas. Como resultado, el primer campo se divide incorrectamente en dos, generando tres valores en lugar de los dos esperados. Esto permite identificar un defecto en el procesamiento de archivos CSV.

Más adelante, se realizará la corrección de este bug para que la función reconozca adecuadamente las comas dentro de campos entre comillas. Posteriormente, se volverá a ejecutar la prueba para verificar que el resultado sea el esperado.

A continuación una foto de la prueba realizada junto a su ejecución.

```python
from unittest.mock import patch
from src.csv_parser import parse_csv_line


@patch(
    "src.csv_parser.csv.reader",
    return_value=iter([["Pérez, Juan", "20"]])
)
def test_parse_csv_line_usa_lector_csv(mock_reader):
    linea = '"Pérez, Juan",20'

    resultado = parse_csv_line(linea)

    assert resultado == ["Pérez, Juan", "20"]
    mock_reader.assert_called_once_with([linea])
```

```
=========================================================== short test summary info ============================================
FAILED test_csv_parser.py::test_parse_csv_line_usa_lector_csv - assert ['"Pérez', ' Juan"', '20'] == ['Pérez, Juan', '20']
```

2.Bug adicional

Como mencionamos anteriormente, al realizar la prueba a la funcion anterior encontramos un bug en la funcion parse_csv_line, esta fue la manera, a modo resumen en la que el bug fue solucionado

Módulo	Descripción	Función	Entrada	Resultado erróneo	Resultado esperado	Corrección aplicada
CSV Parser	La función separa por cada coma sin respetar las comas dentro de campos entre comillas.	parse_csv_line()	'"Pérez, Juan",20'	['"Pérez', ' Juan"', '20']	['Pérez, Juan', '20']	Se reemplaza linea.split(',') por next(csv.reader([linea])), para interpretar correctamente los campos CSV entre comillas.

3.Workflow

Se creó el archivo pytest.yml dentro de la carpeta .github/workflows para automatizar la ejecución de las pruebas unitarias. Se configuró el evento push, de modo que cada vez que se envían cambios al repositorio, GitHub Actions inicia el proceso automáticamente. El pipeline utiliza una máquina virtual con Ubuntu, descarga el código del repositorio, configura Python 3.13 e instala las dependencias definidas en requirements.txt. Finalmente, ejecuta pytest para comprobar el funcionamiento de las pruebas y detectar posibles errores en el código. Esta integración facilita la validación continua del proyecto sin necesidad de ejecutar las pruebas manualmente después de cada actualización.

```yaml
name: ejecturar pytest

on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.13'

      - run: pip install -r requirements.txt
      - run: pytest
```

Reflexión final

La cobertura de código es una métrica cuantitativa indispensable, pero no cualitativa. Registrar un porcentaje elevado indica únicamente qué líneas del código fuente fueron ejecutadas durante las pruebas, pero no garantiza por sí sola que el software sea totalmente correcto ni que esté libre de fallos lógicos.

En concreto, la cobertura de código no mide:

1. La calidad y pertinencia de las aserciones (Assert): Una prueba podría recorrer el 100% de las instrucciones de una función sin verificar adecuadamente sus resultados o haciendo comprobaciones triviales, lo que genera una falsa sensación de seguridad.
2. Casos límite o escenarios no considerados: La cobertura muestra si la línea existente se ejecutó, pero no detecta la falta de validaciones ante entradas imprevistas, valores nulos, estructuras vacías o condiciones de borde no contempladas en el diseño original.
3. La corrección de la lógica de negocio: Si el requerimiento exige una regla de negocio y el código implementa una fórmula equivocada, las pruebas pueden cubrir el 100% del código ejecutable y aún así dar por válido un comportamiento funcionalmente incorrecto.

El desarrollo de esta actividad me permitió poner en práctica lo que hemos visto en clase y me gustaria hablar un poco de cada parte

- Estructura y mantenibilidad con el patrón AAA: La adopción del esquema Arrange-Act-Assert da mas claridad conceptual a cada prueba unitaria, facilitando la identificación inmediata de las condiciones iniciales, la acción evaluada y el resultado esperado.
- Mentalidad de detección de errores: Asumir el rol de detective para diseñar pruebas enfocadas en romper las funciones me hizo pensar bastante.. Confirmar un test en rojo antes de corregir el código demostró que una prueba fallida es una evidencia de hallazgo y no un fracaso del proceso.
- Aislamiento mediante Mocking: La implementación de la prueba del csv parser me tuvo un rato tratando de hacerla, pero realmente me parece muy valioso poder separar de esa manera la logica de los datos reales, para no depender siempre de una entrada que no este
- Workflow: La integración de GitHub Actions para ejecutar pytest automáticamente en cada evento de push realmente fue la parte que mas me gusto por que pude ver como realmente lo que vemos en clase funciona en un caso real.